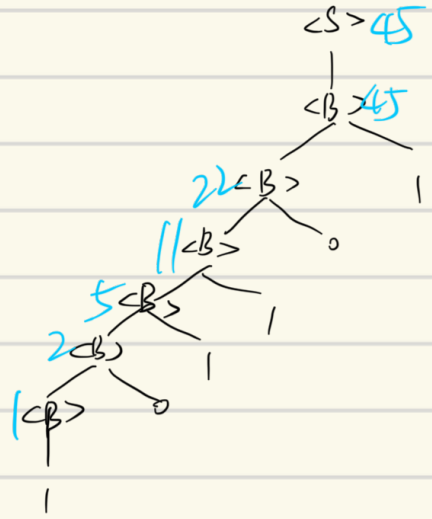
1. 设有 SDD 如下：

产生式	语义规则
S → B#	S .val=B.val
B → B 0	B .val =2×B_1 .val
B → B 1	B .val =2×B_1 .val+1
B → 1	B .val =1

给出 101101 的注释语法分析树。



2. 含小数点的二进制数文法定义如下：

S→L.L|L

L→LB|B

B→0|1

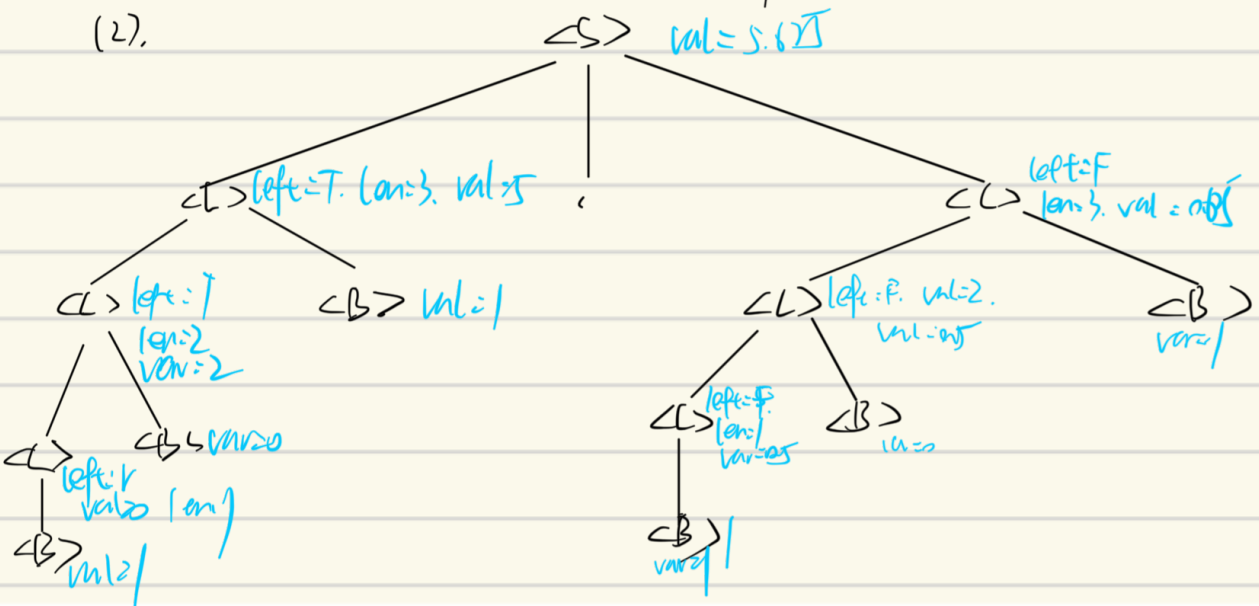
(1)根据上述文法设计一个 SDD，实现二进制数到十进制数的翻译，比如 101.101 翻译为 5.625。

(2)给出 101.101 翻译为 5.625 的注释语法分析树。

(1).

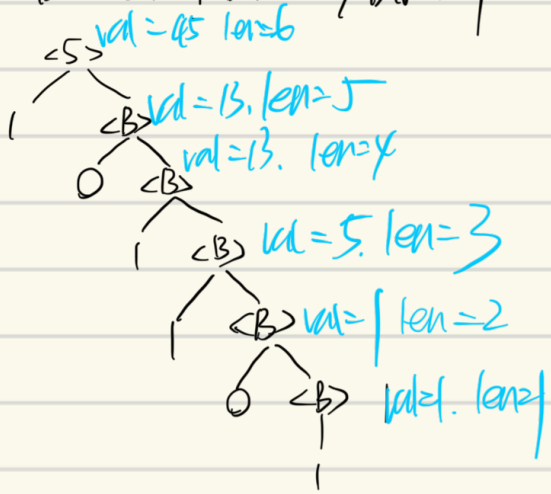
产生式	语义规则 (in:left co: val, len)
S→L1.L2	in: L1.left=T L2.left=F. co: S.val=L1.val+L2.val.
S→L	in: L.left=T. co: S.val=L.val
L→L1B	in: L1.left=L.left co: L.len=L1.len+1. L.val=(L.left)?:(L1.val×2+B.val):(L1.val+B.val×2^{len})
L→B	in: co: L.len=1. L.val=(L.left)? B.val : B.val+2
B→0	in: co: B.val=0
B→1	in: co: B.val=1

(2).



3. （选做）在保证语义相同的情况下，改写 1 题中的 SDD，使文法不是左递归的。

产生式	语法规则
S->1B#	
B -> 0B	B.len+=1
B -> 1B	B.val+=B.val×$2^{B.len}$ B.len+=1
B -> 01	B.val=0 / B.val=1 B.len=1



4. 设包含有数组引用的复制表达式的 SDD 如下：

产 生 式	语 义 规 则
S → i=E;	gen(= E.place - i)
S → L=E;	gen([]= E.place - L.base[L.addr])
E → E+E	E.place=newtemp();gen(+ E_1 .place E_1 .place E.place)
E → i	E.place = i
E → L	E.place=newtemp();gen(=[] L.base[l.addr] - E.place)
L → i[E]	L.base=i;L.addr=newtemp();gen(* E.place L.elem.width L.addr)
L → L[E]	L.base=L_1.base;L.type=L_1.type;t=newtemp();L.addr=newtemp();gen(* E.place L.elem.width t); gen(+ L_1.addr t L.addr)

给出下列赋值语句的四元式代码：

(1) x=a[i][j]+b[i][j]

(2) a[b[i][j]][c[k]]=a[i][j]

说明：

a) gen()是产生四元式代码的函数，参数依次为操作符、第一操作数、第二操作数、计算结果；

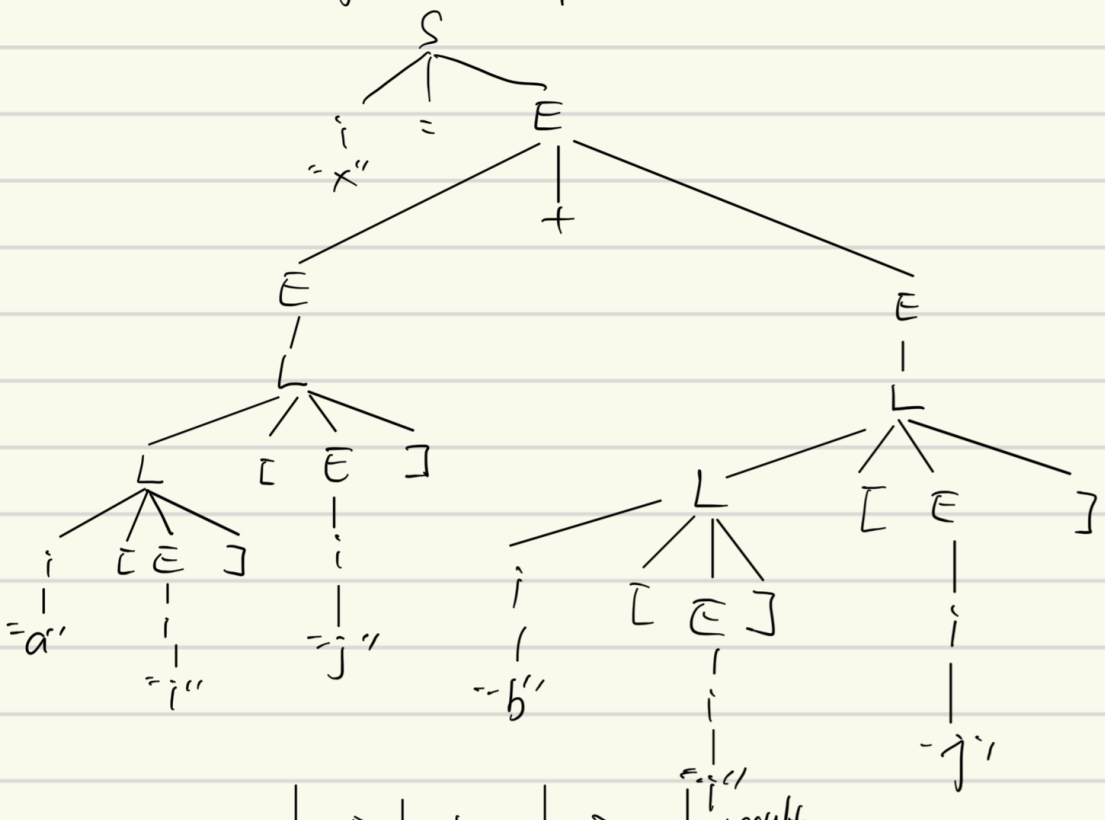
b) newtemp()是临时变量申请函数；

c) 关于数组中用到的属性 width 的 SDD 如下：

产 生 式	语 义 规 则
D → B iC	w=B .width；i.width=C.width
B → int	B .width =4
B → float	B .width =8
C → ε	C.width=w
C → [num]C	C.width=num.value×C_1.width;C.elem.width=C_1.width

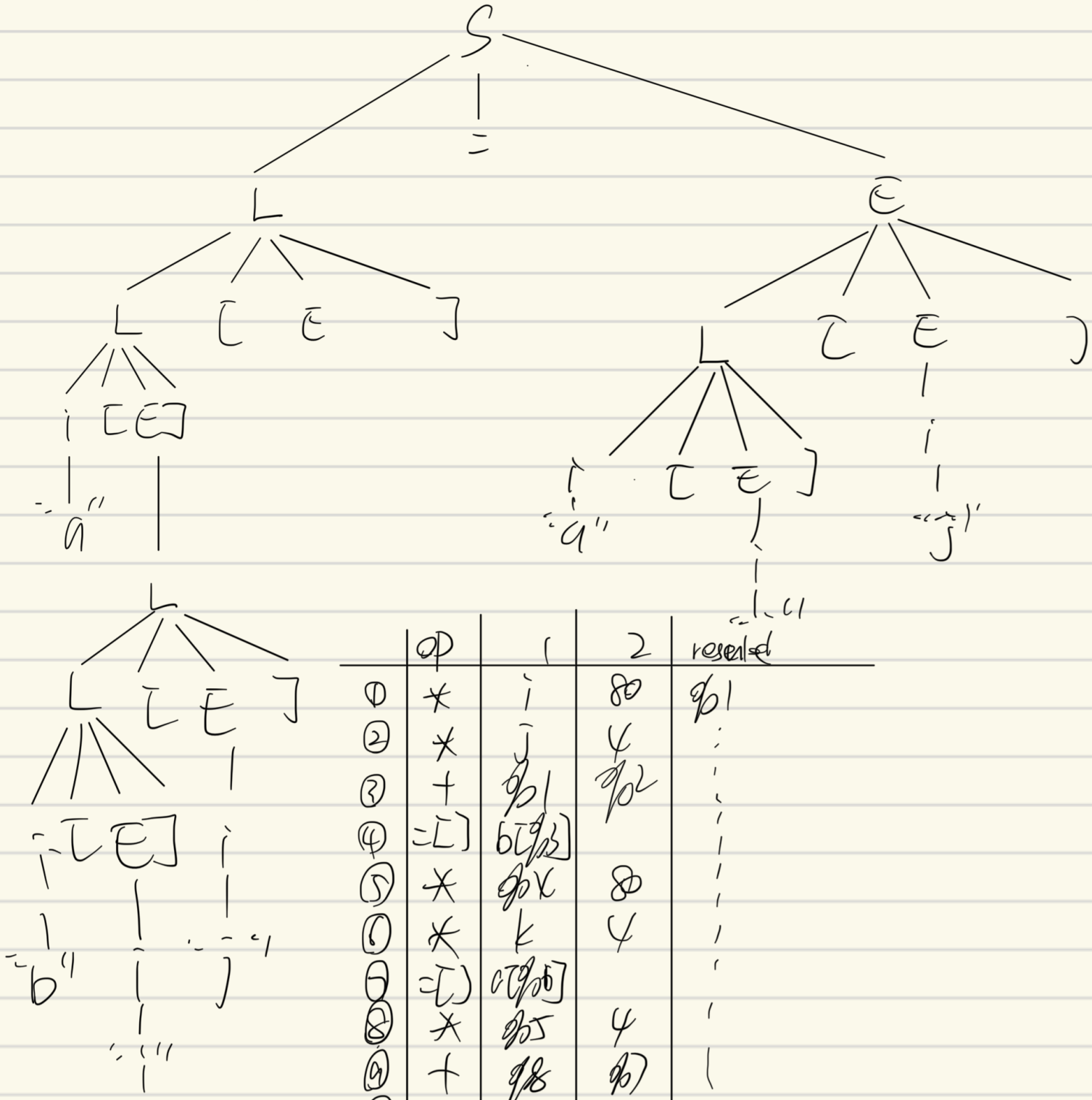
d)语句中的数组说明：int a[10][20],b[10][20];c[10]。

(1), x = a[i][j] + b[i][j]



	op	1	2	result
①	*	i	80	p1
②	*	j	4	p2
③	+	p1	p2	p3
④	=[]	a		p4
⑤	*	i	80	p5
⑥	*	j	4	p6
⑦	+	p5	p6	p7
⑧	=[]	b[p7]		p8
⑨	+	p4	p8	p9
⑩	=	p9		x

(2). a[b[i][j]][c[k]] = a[i][j]



	op	1	2	reserved
①	*	i	80	%1
②	*	j	4	
③	+	%1	%2	
④	=[]	b[%3]		
⑤	*	%4	80	
⑥	*	k	4	
⑦	=[]	c[%6]		
⑧	*	%5	4	
⑨	+	%8	%7	
⑩	*	i	80	
⑪	*	j	4	
⑫	+	%10	%11	
⑬	=[]	a[%9]		%13
⑭	[]=	%13		a[%9]

5.有SSD

产生式　　语义规则

B→E	B.true=makelist(nextinstr); B.false=makelist(nextinstr+1); gen(J_T E.PLACE - 0); gen(J_F E.PLACE - 0)
B→E= =E	B.PLACE=NEWTEMP； gen(= = E1.PLACE E2.PLACE B.PLACE); B.true=makelist(nextinstr)
	B.false=makelist(nextinstr+1); gen(J_T B.PLACE - 0); gen(J_F B.PLACE - 0)
B→!B	B.true=B_1.false; B.false=B_1.true
B→(B)	B.true=B_1.true; B.false=B_1.false
B→B||MB	backpatch(B_1.false,M.instr); B.true=merge(B_1.true,B2.true); B.false=B_2.false
B→B&&MB	backpatch(B_1.true,M.instr); B.true=B_2.true; B.false=merge(B_1.false,B_2.false)
S→if(B)MSNelseMS	backpatch(B.true,M1.instr); backpatch(B.false,M_2.instr); S.next=merge(S_1.next,N.next,S_2.next)
S→whileM (B) MS	backpatch(S_1.next,M1.instr); backpatch(B.true,M_2.instr); S.next=B.false;gen(J - - M_1.instr)
M→ε	M.instr=nextinstr
N→ε	N.next=makelist(nextinstr); gen(J - -0)
S→{L}	S.next=L.next
S→i=E；	S.next=null
L→LMS	{backpatch(L1.next, M.instr); L.next=S.next;}
L→S	L.next=S.next

SDD 的说明：

a)四元式编号作为跳转地址使用

b)函数 makelist(x)表示出现了标号的引用时不知道地址的情况，函数参数 x 为标号的使用位置，单遍处理生成中间代码需要使用拉链—返填技术，假设通过标号表记录相关信息，此处为标号的引用出现，按标号引用记录入标号表。

c)函数 backpatch(x,y)表示标号 x 的定义位置在 y,需要返填标号 x 的使用位置。

d)函数 merge(x,y,z,….)表示标号 x,y,z…合并为一个标号

e)nextinstr 表示下一条四元式的编号

有程序段：

```
if(a==b&&(c==d||e==f))
  {while(!(x==y))
          x=x+y;
          s=x;}
 else
    s=y;
 z=s;
```

问题：

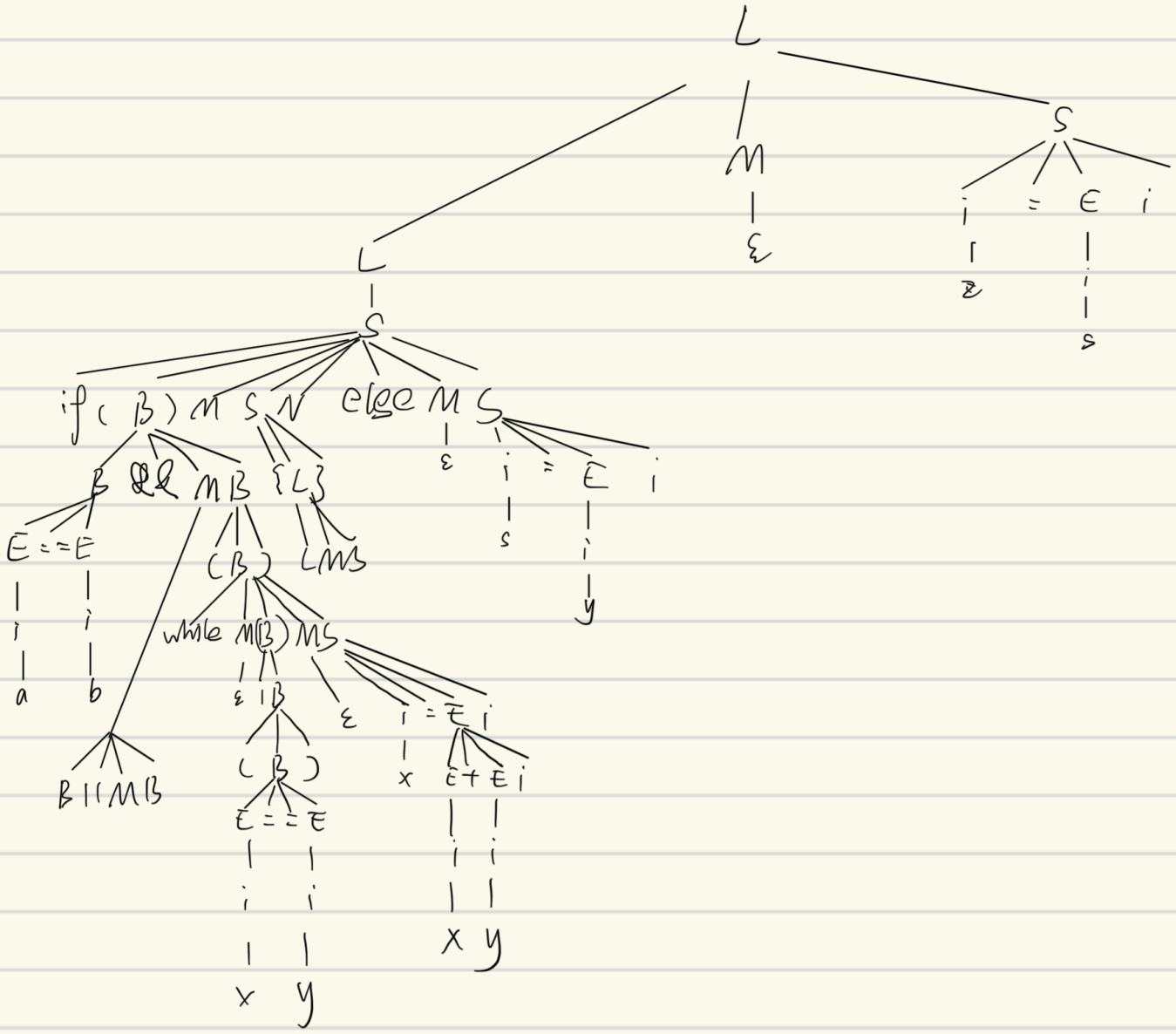
(1)给出该程序段的语法分析树(表达式 E 结构使用 4 题的)。

(2)给出该程序段语义处理后的四元式形式的目标代码（代码中的表达式 E 结构使用 4 题的 SDD）。

(3)给出该程序段语义处理后的拉链-返填使用的标号表。

标号表按以下示例内容填写 ：

标号名	定义否（1/0）	返填顺序
L_i	1	⑤→②→①
…		


L
M
ε
S
i
z
=
E
i
s
L
S
if (B) M S N else M S
ε
i
s
=
E
i
y
B && M B
{ L }
E == E
i
a
i
b
B || M B
(B)
L M S
while M (B) M S
ε
(B)
E == E
i
x
i
y
ε
i
x
= E i
E + E i
i
x
i
y

(2)

标号	op.	1	2	result
①	==	a	b	101
2	JT	101		4
3	JF	101		18
L1 4	==	c	d	102
5	JT	102		10
6	JF	102		7
L4 7	==	e	f	103
8	JT			10
9	JF			18
L3 10	==		y	104
11	JT			16
12	JF			13
L6 13	+	x	y	105
14	=	105		x
15	J			10
L5 16	=	x		5
17	J			19
L2 18	=	y		5
L7 19	=	z		5

(3)

标号名	定义否	退填顺序
L1	1	④ → ②
L2	1	⑱ → ⑨ ③
L3	1	⑩ → ⑧ → 15 → ⑮
L4	1	⑦ → ⑥
L5	1	⑯ → ⑪
L6	1	⑬ → ⑫
L7	1	⑲ → ⑰